

2. Código fuente, objeto y ejecutable

Modelos de traducción · Compilación · Máquinas virtuales · Empaquetado · Java desde consola

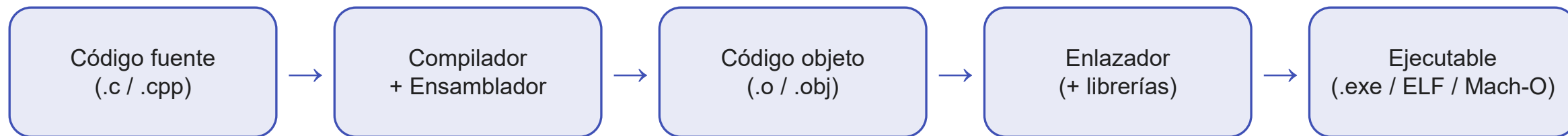
2.1 Del código fuente al binario: los tres modelos de traducción

Un ordenador solo entiende ceros y unos. Los lenguajes que usamos (C, Java, Python...) son legibles para personas, pero la CPU no los entiende: hace falta traducción. Según el lenguaje, hay tres caminos:

Modelo	Cómo funciona	Flujo	Ejemplos
Compilado	El compilador traduce TODO el programa de una vez antes de ejecutarlo. El resultado es un ejecutable independiente con instrucciones binarias para la CPU.	Fuente (.c) → Compilador → Código objeto (.o) → Enlazador → Ejecutable (.exe, ELF)	C, C++
Interpretado	Sin compilación previa: el intérprete lee el código fuente LÍNEA A LÍNEA al ejecutarlo, traduciendo y ejecutando sobre la marcha.	Fuente (.py, .js) → Intérprete → Ejecución directa	Python, JavaScript

Analogía del libro: compilado = traductor que traduce todo el libro antes de publicarlo · interpretado = intérprete simultáneo frase a frase · JIT = intérprete que memoriza las frases que más se repiten.

2.2 El proceso compilado en detalle



- **Compilación:** traduce el fuente a código objeto. Contiene binario, pero incompleto: sabe que llamará a printf, pero no en qué dirección de memoria estará. Esas referencias quedan pendientes.
- **Ensamblado:** convierte las instrucciones al formato exacto de la CPU. En la práctica va integrado en el compilador, en un solo paso.
- **Enlazado:** el linker une todos los archivos objeto y las librerías (bloques de código ya compilado que el programa reutiliza, como las funciones matemáticas o de E/S), resuelve las referencias pendientes y produce el ejecutable final.
- El formato del ejecutable depende del SO: .exe en Windows, ELF en Linux, Mach-O en macOS. Por eso un .exe de Windows no funciona directamente en Linux.

Enlazado	Qué incluye	Ventaja	Inconveniente
Estático	Todo el código de las librerías dentro del ejecutable	Funciona solo, sin dependencias	Más grande
Dinámico	Referencias a librerías externas (.dll en Windows, .so en Linux)	Más ligero	Falla si la librería no está instalada

El clásico error de las DLL: "no se encontró MSVCP140.dll" — el equipo destino no tiene la versión correcta de una librería dinámica. Por eso los instaladores incluyen redistribuibles de Visual C++.

2.3 Comparativa y cuándo elegir cada modelo

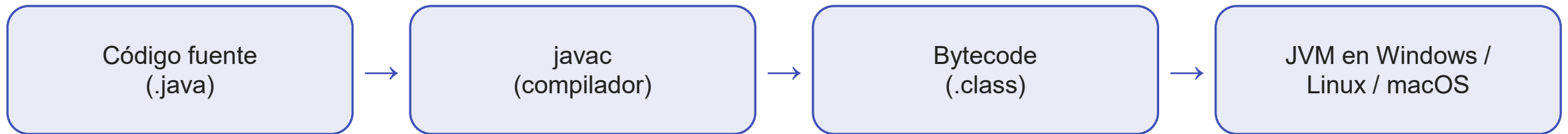
Tipo	Cómo funciona	Ejemplo	Ventajas	Inconvenientes
Compilado	Todo a código máquina antes de ejecutar	C, C++	Muy rápido; el compilador optimiza	Recompilar tras cada cambio; el binario solo vale para un sistema
Interpretado	Ejecuta línea a línea en tiempo real	Python, JS	Flexible; multiplataforma; desarrollo rápido	Más lento; necesita el intérprete instalado
JIT (VM)	Bytecode + VM que traduce en ejecución	Java, C#	Portabilidad + buen rendimiento; se adapta a cada máquina	Necesita la VM; arranque algo más lento

	Compilado (C/C++)	Interpretado (Python/JS)	JIT (Java/C#)
Elige cuando...	El rendimiento es crítico y el sistema destino es fijo	Necesitas desarrollar y cambiar rápido	Necesitas portabilidad y buen rendimiento a la vez
Casos típicos	Videojuegos, SO, drivers, firmware	Scripts, web, prototipos, ciencia de datos	Apps empresariales, Android, backend con carga
Punto fuerte	Velocidad de ejecución máxima	Ciclo editar-probar sin recompilar	Funciona en cualquier sistema con la VM

2.4 Máquinas virtuales y portabilidad del bytecode

Al compilar Java no obtienes un ejecutable para Windows o Linux: obtienes BYTECODE, un formato intermedio que cualquier sistema ejecuta si tiene instalada la JVM.

Analogía: un PDF. No importa dónde lo creaste — cualquier ordenador con un lector de PDF lo abre. El PDF es un formato intermedio universal; el bytecode de Java funciona igual.



El mismo .class funciona en los tres sistemas: "Write Once, Run Anywhere" (WORA), la promesa de Java.

¿Cómo funciona el JIT?

La JVM empieza interpretando el bytecode instrucción a instrucción. Si detecta que una parte se ejecuta muchas veces (un bucle muy repetido), la compila a código máquina nativo: las próximas veces va a velocidad de binario nativo.

Ecosistema	Compilador → intermedio → ejecutor
Java / JVM	javac genera .class con bytecode → la JVM lo ejecuta y el JIT optimiza las partes críticas (descarga: adoptium.net)
C# / CLR	El compilador de C# genera CIL (Common Intermediate Language) → la CLR de .NET lo ejecuta con JIT; .NET moderno es multiplataforma

AOT y 2.5 Empaquetado y distribución

AOT (Ahead-of-Time) — el camino contrario al JIT: compila todo ANTES de ejecutar y genera un binario nativo. Se pierde portabilidad pero se gana arranque rápido y no se necesita la VM. Herramientas: GraalVM (Java), NativeAOT (.NET).

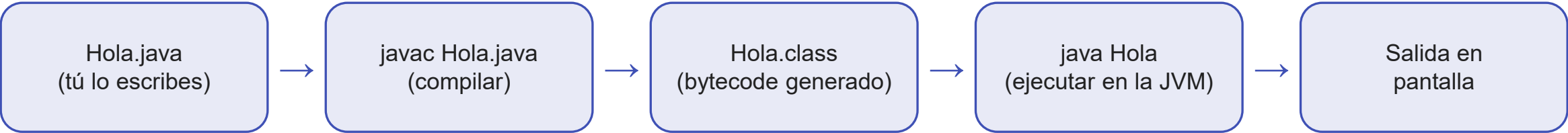
Cómo llega el programa al usuario

Método	Qué incluye	Cuándo usarlo	Ejemplos
Instalador	Solo el programa (las librerías pueden descargarse durante la instalación)	Apps de escritorio tradicionales	.exe / .msi (Windows), .deb (Ubuntu), .rpm (Fedora)
Bundle	Programa + todas las dependencias en un paquete portable	Apps multiplataforma sin instalación de librerías	Electron (VS Code, Discord): incluye Chrome y Node.js dentro
Contenedor	Programa + dependencias + entorno completo (SO, variables, red)	Servicios web, despliegues en la nube	Docker: igual en el portátil, en pruebas y en producción

Ejemplos concretos: videojuego de PC → instalador .exe que descarga DirectX si falta · app Android → archivo .apk (Android Package) con bytecode + recursos · servicio web → contenedor Docker que el servidor arranca en segundos.

2.6 Compilar y ejecutar Java desde consola

Dos pasos obligatorios y separados — no puedes saltarte ninguno:



Comando	Qué hace	Detalle
javac Hola.java	Compila: comprueba la sintaxis y genera el bytecode	Si todo va bien no imprime nada; aparece Hola.class junto al .java
java Hola	Ejecuta: arranca la JVM con esa clase	Se escribe el NOMBRE DE LA CLASE, sin extensión

Error 1 — el nombre del archivo debe coincidir con la clase: public class Hola → el archivo TIENE que llamarse Hola.java (mayúsculas incluidas), o javac da error.

Error 2 — no escribas java Hola.class: java espera el nombre de la clase; con la extensión buscará una clase llamada literalmente "Hola.class" y fallará (Could not find or load main class).

Paquetes: en proyectos reales las clases se agrupan en paquetes, que son carpetas. Un paquete org.entornos equivale a la ruta org/entornos/, y Java exige que la estructura de carpetas coincida exactamente con la declaración package del archivo.